Software Project Management

Week 12 Submission

G5 Team 7:
Jenis | Chelsea Dias | Chelsea Goh | Haroun | David | Wilfred

*Refer to Project Playbook for more details

How we conducted our scrum process*

Timeline

1. **Sprint 0 (Setup Sprint):** 8 Sep - 17 Sep **(10 Days | Week 4 - 5)**
2. **Sprint 1:** 18 Sep - 29 Sep **(12 Days | Week 5 - 7)**
3. **Sprint 2:** 2 Oct - 13 Oct **(12 Days | Week 7 - Week 9)**
4. **Sprint 3:** 16 Oct - 27 Oct **(12 Days | Week 9 - Week 11)**

- 4 Days is due to other module commitments
- Compensated through async updates & check-ins through Telegram Group Chat

Tools Used

- **Google Sheets & Google Docs:** Sprint Documentations, Product & Sprint Backlog, Burndown Chart, Testing Documentation
- **GitHub:** Repo, Git Branching, CI (GitHub Actions)
- **Zoom + MSTeams + Telegram:** Online Meetings + Communications
- **Figma + PlantUML:** Wireframing + C4

Throughout Sprint

- Unit Tests
- Continuous Testing

Standups (Every 4 Days)

Discussed
- Status Update
- Blockers

Sprint 12 Days

Sprint Planning (1st Day of Sprint)

Sprint Review + Retrospective (Last Day of Sprint)

Retro Discussed
- What went well
- What did not go as expected
- Improvement for Next Sprint

- Update Project Playbook
- Product Backlog
- Craft User Stories & Acceptance Criteria

- Confirm User Stories & Acceptance Criteria
- Estimation & Planning Poker
- Sprint Backlog & Priority
- Establish Sprint Goal

- Demo + UAT
- Marked as “Completed” if Definition of Done (DoD) is met
- Velocity + Burndown

SMU Singapore Management University | School of Computing and Information Systems

*Refer to Project Playbook for more details

Key changes to the scrum process (from the first sprint to the last)*

Legend:
- Sprint 0
- Sprint 1-3
- No changes
- □ Result

Backlog & User Stories Creation

Initial backlog created; some stories were larger than 8 points, identified as epics and we split them.	Stories consistently ≤ 8 points, making them easier to estimate, plan, and complete within a sprint.

Breaking down larger stories improved predictability, balanced workload, and ensured more stories could be completed within the sprint.

Sprint Structure

Sprint Duration: 10 Days foundational setup of Scrum routines	**Sprint Duration:** 14 Days standardized with clear goals & capacity planning

Standardizing sprint length improved predictability, allowed better resource planning, and gave the team a consistent rhythm for delivery.

Sprint Planning, Definition of Done

Both DoR and DoD remained unchanged throughout the project, as they were clearly defined and effective from the start.
DoR: Ensured every user story was clear, estimable (≤8 points), and testable before sprint inclusion, maintaining sprint predictability.
DoD: Provided a shared understanding of completion standards across code quality, testing, design compliance, and deployment.

There is no change. Why this consistency: The existing definitions already supported transparency, quality assurance, and smooth sprint flow, so no revision was needed.

Tools & Collaboration

Used Google Sheets, Google Docs, Telegram, Physical Meetings and GitHub for planning, documentation, and communication.	Introduced branching in GitHub and introduced Zoom meetings

Implementing branching enhanced collaboration, reduced merge conflicts, and made it easier to track progress and changes in code. Implemented Zoom enables everyone to meet busy schedule.

Meetings & Communication

Stand-ups held twice per sprint to coordinate work, physical meetings only	Daily communication via Telegram plus structured stand-ups via Zoom + MS Teams meetings, enabling quicker updates and smoother coordination

More frequent and flexible communication helped surface blockers earlier, accelerate decision-making, and maintain smooth coordination among team members.

Sprint Review & Retrospective

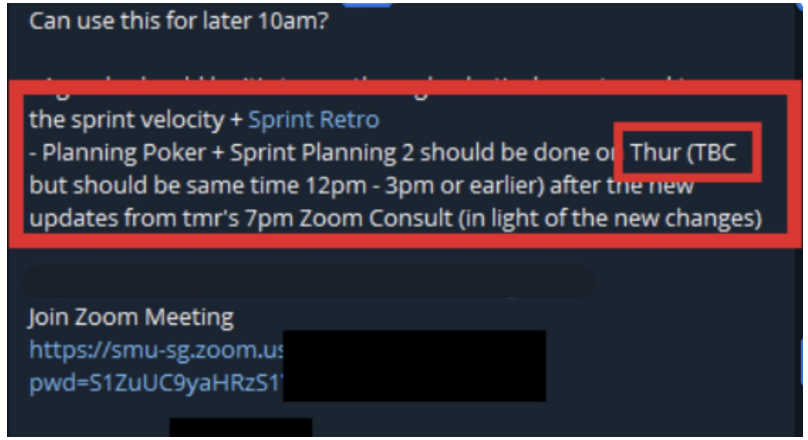
Structure stayed consistent throughout all sprints.
Reviews: Demoed completed stories, recorded Product Owner feedback, and re-added unfinished items to backlog.
Retrospectives: Reflected on improvements after each sprint.

Formats were already effective for transparency and continuous improvement.

How do we manage changes to our Product backlog

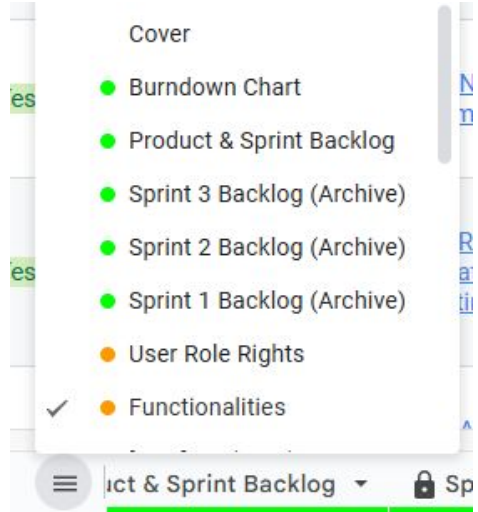
Initial creation of Product backlog:

- Product Backlog is maintained as a living document in Google Sheets.
- The initial Product Backlog was created during Sprint 0, based on client requirements and clarification sessions.
- Stories were drafted using a template with clear titles, acceptance criteria. Estimation of story points are done together during planning poker.



Managing changes to Product backlog:

- Changes (new features, updated requirements, reprioritization) are raised by the Product Owner (PO) or during refinement sessions.
- The Scrum Master schedules refinement or discussion meetings when major updates arise.
- Stories are updated with revised Acceptance Criteria, story points, and dependencies.
- Estimation is done using Planning Poker to ensure consistency and valid effort estimates.
- Approved changes are reflected in the backlog by Sprint Planning session.
- After each sprint (Sprint Retrospective), we archive a snapshot of the Product Backlog. Since the backlog is a living document, this makes it easier to track changes, monitor progress, and ensure transparency.



How we handled the customer's change of requirements

4	Yes	Task ...	[TM] Task Management	new tasks, view their current work, ils, and remove items when longer needed. Tasks can include assigned team members, and This forms the foundation for m productivity.	Users can **create new tasks/subtasks (can be in a project or standalone), view their current tasks/subtasks, update task/subtask details,** and **update their statuses**. Tasks can include deadlines, notes, invited collaborators, and status tracking. **Managers** and above **can assign tasks/subtasks to their staff, transferring ownership to them**. This forms the foundation for personal and team productivity.
5	Yes	Task ...	[TGO] Task Grouping & Organisation	uped into meaningful categories departments, or themes. This organized by seeing work in ng navigation, and enabling focus tives.	**Users can create projects** to house their tasks and subtasks. **Collaborators can be invited to work on projects**. This ensures that there are proper organisation and navigation for projects, as **most staff work on multiple**.

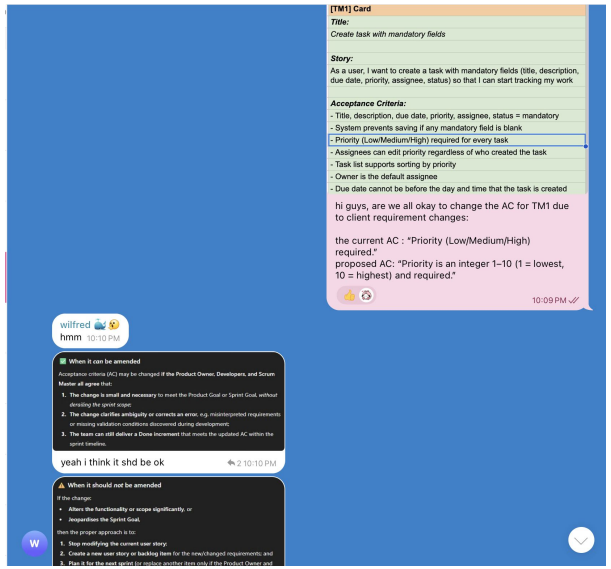
Users can **create new tasks/subtasks (can be i project or standalone), view their current tasks/subtasks, update task/subtask details, and update their statuses**. Tasks can include deadlines, notes, invited collaborators, and status tracking. **Managers** and above **can assign tasks/subtasks to their staff, transferring ownership to them**. This forms the foundation for personal and team productivity.

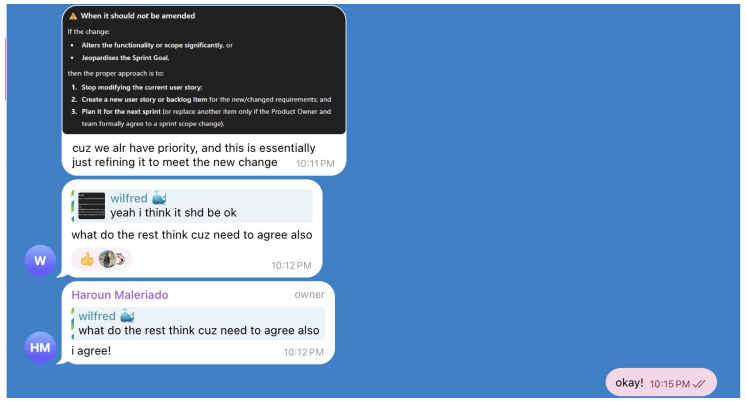
Task recurrence allows users to **create tasks that automatically repeat at set intervals**, reducing the need to manually recreate routine work. **When a recurring task is completed, the system generates the next instance with an updated due date**.

Users can create projects to house their tasks and subtasks. **Collaborators can be invited to work on projects**. This ensures that there are proper organisation and navigation for projects, **as most staff work on multiple**.

Tasks are also assigned to a **numbered Priority Bucket from 1 – 10 (with 1 being the least important and 10 being the most), organising them by their level of importance**. This ensures that critical work is clearly distinguished from less urgent tasks. **Priority must be set at time of creation** but **can be changed by the owner later**.

142	**[TM12] Card**	**5**		TM12 Test Cases
143	***Title:***			
144	*Create recurring tasks*			
145				
146	***Story:***			
147	- As a staff user, I want to create recurring tasks (daily/weekly/monthly) so that I don't duplicate them manually			
148				
149	***Acceptance Criteria:***			
150	- System generates new task instances per recurrence interval			
151	- Each recurrence is logged as a separate task			
152	- Completing one recurrence does not affect future recurrences.			
153	- When a recurring task is completed, the system generates the next instance with an updated due date.			
154	- Attached file will be linked / duplicated for recurring task		TM060	
155	- Comments do not need to be recurred			





- Documented the change in our sprint management documentations
- Added the user story for recurring tasks which had no user story yet and added it to the sprint 2 backlog
- For the user story on priority of tasks: with approval from product owner, scrum master, and developers, changed the AC mid- sprint as the scope of the sprint would not change
  - This is because scrum allows flexibility to adjust in-progress stories if scope changes are manageable.
  - The change was essential for subsequent sprint work, and completing it early prevented downstream delays.

Acceptance Criteria:
- Title, description, due date, priority, assignee, status = mandatory
- System prevents saving if any mandatory field is blank
- Priority (Low/Medium/High) required for every task

Acceptance Criteria:
- Title, description, due date, priority, assignee, status = mandatory
- System prevents saving if any mandatory field is blank
- Priority (1 being the least priority - 10 being the highest priority) required for every task

*Diagrams built using PlantUML at https://www.planttext.com/

Rationale for the system's architecture & design

Software Architecture: Modular Monolith (with background workers)

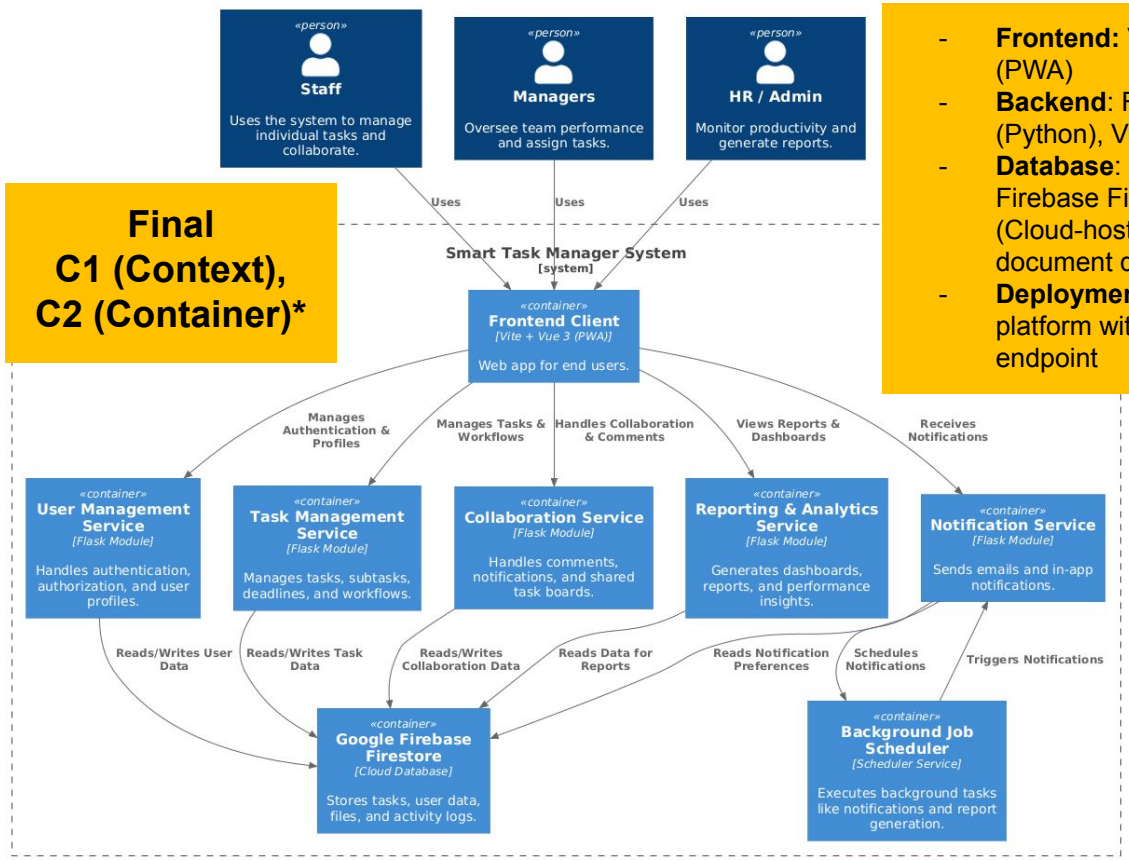
Rationale:

- Keeps operational complexity low **(performance)** while allowing for clear module boundaries for the 1st release
- Can scale the background worker horizontally as users grow **(scalability)**

How C4 Helped:

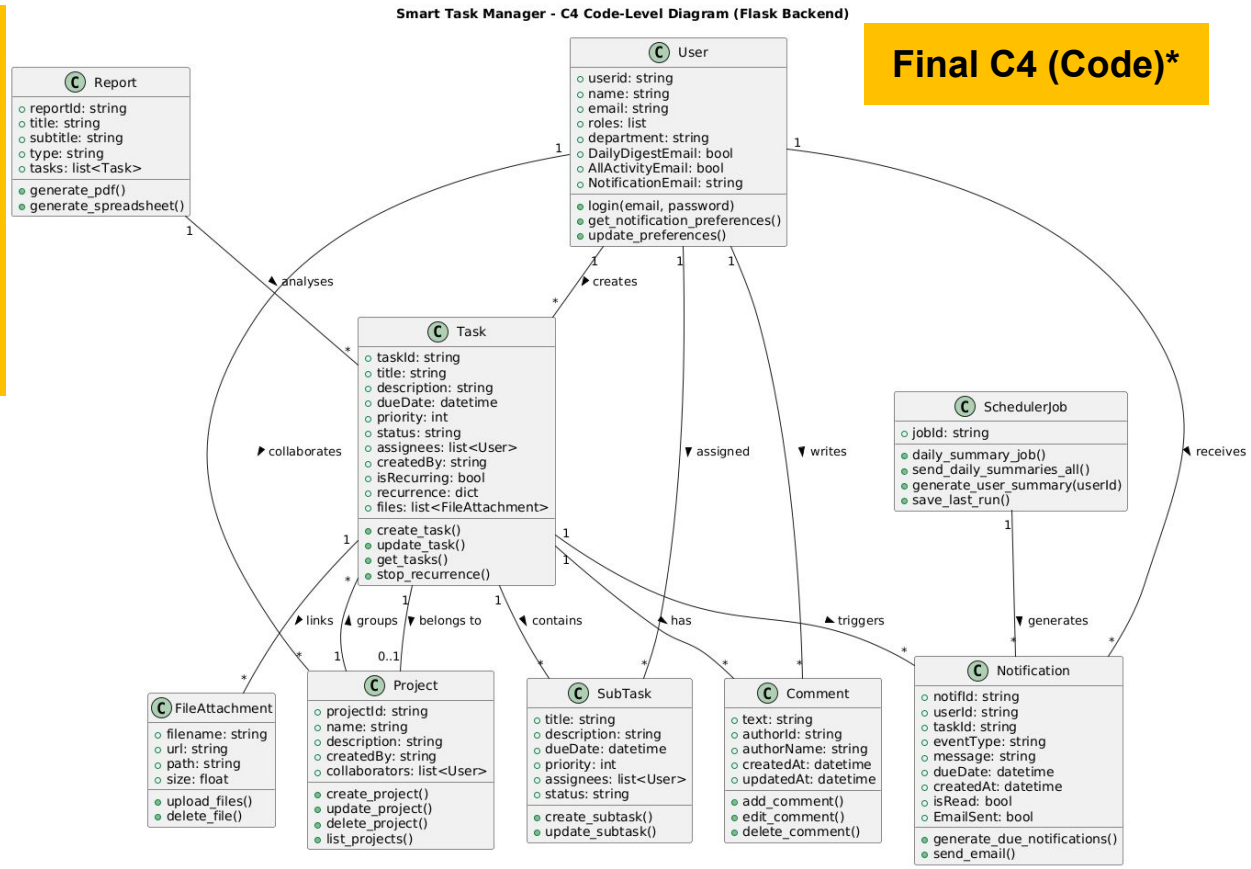
- **Shared Understanding:**
  - Provided the team a shared "blueprint" for coding
  - Diagrams make it clear the modular monolith + worker choice
- **Improved Traceability:**
  - Each module in C3 maps directly to a set of classes in C4, ensuring consistent understanding between architecture & implementation

Final C1 (Context), C2 (Container)*

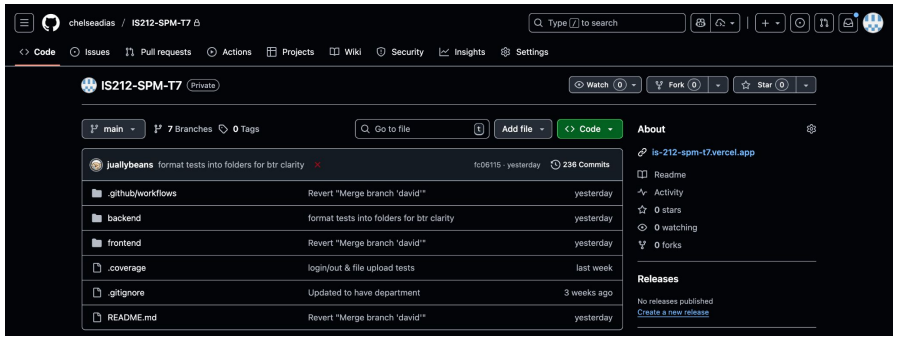


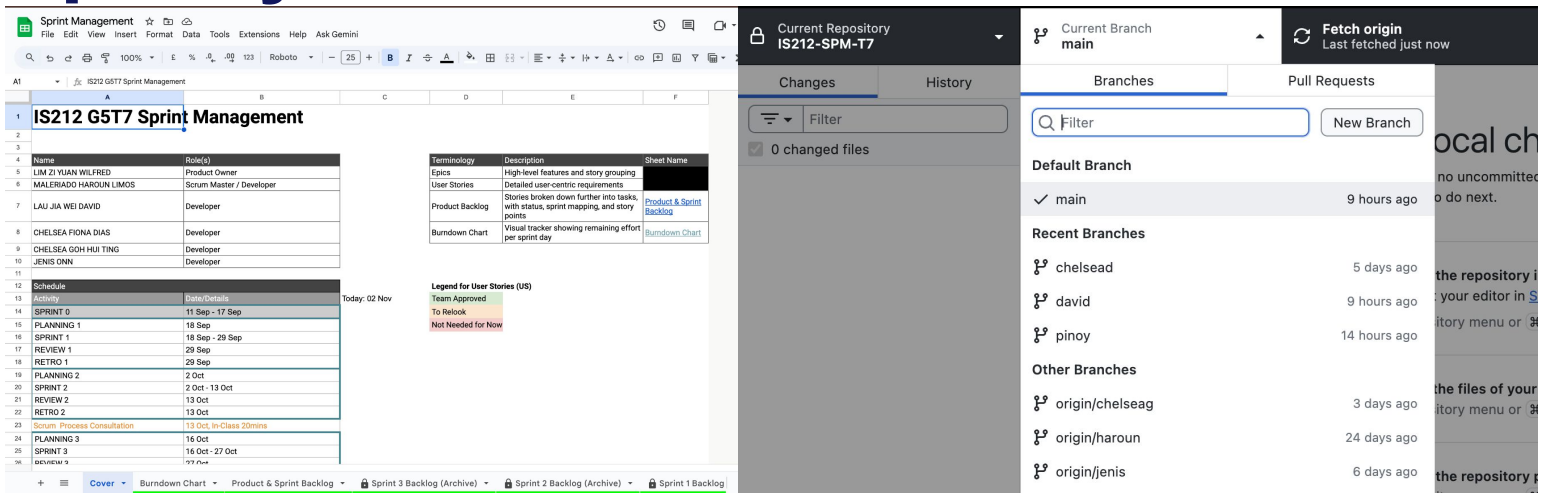
- **Frontend:** Vite + Vue 3 (PWA)
- **Backend**: Flask (Python), Vue.js
- **Database**: Google Firebase Firestore (Cloud-hosted NoSQL document database)
- **Deployment:** Cloud platform with HTTPS endpoint

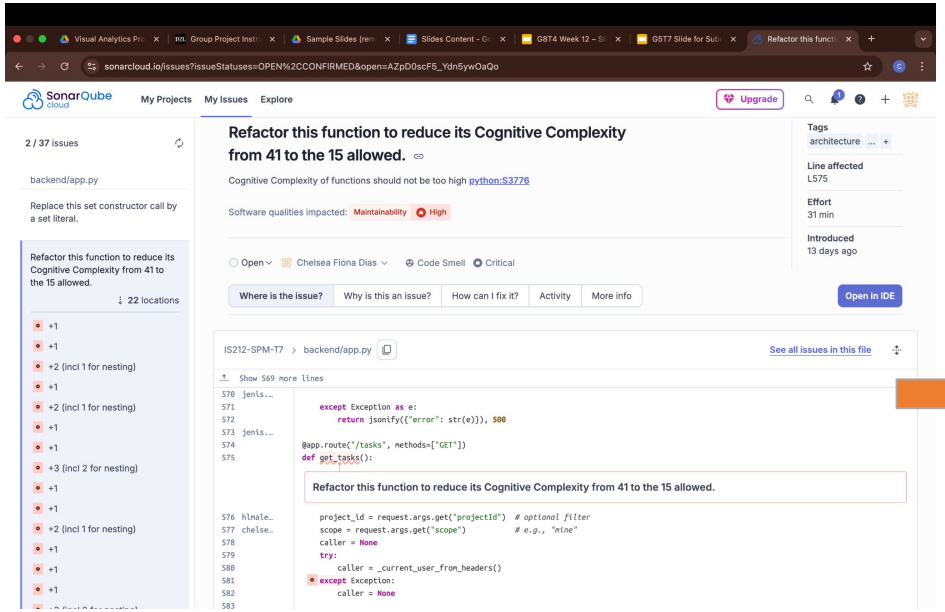
Final C4 (Code)*



How we ensured a high-quality codebase









- Made use of Github and Google Sheets for collaboration
- Firebase for our DB, Vue.js and Python Flask for code
- Arranged components according to pages, and the pages were built from the user stories
- Used SonarQube to refactor code smells
- Made use of Figma to have a consistent UI base
- Peer code reviews for quality and consistency
- We used branching to avoid merge conflicts

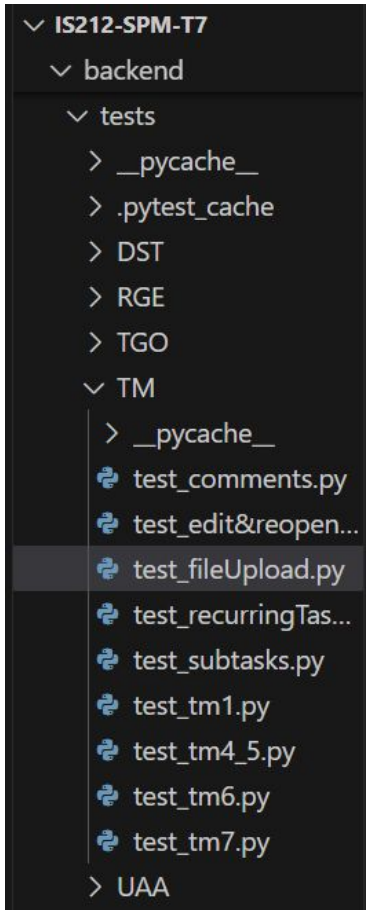
How we tested our system

NavPage / Template	[RGE1] Card
Test Case ID:	**RGE1-T1**
Acceptance Criteria:	HR/Admin can generate report for any project.
Items	**Description**
Test Scenario	(Positive) Valid Admin Role Access.
Pre-conditions	- Admin (julia@example.com) is logged in - Project 123 exists
Test Steps	- Navigate to Tasks page (click Tasks on the Navbar) - Click the Generate Report button - Select the relevant field data
Test Data	Report Type: Project Report Select Project: 123
Expected Result	- The 'Generate Report' button is visible and clickable. - The 'Project Report' loads successfully under the fields, displaying the data for Project 123.
Actual Result	Same as expected
Pass/Fail/Not Executed/Blocked	Pass
Remarks	
Created By*	Jenis
Date of Creation*	24 October
Executed By*	Jenis
Date of Execution*	24 October

Our testing process ensured that every system feature was validated for correctness, reliability, and alignment with user requirements.

Functional Testing

- Each user story in our backlog is mapped to functional test cases compiled in a google spreadsheet, ensuring full traceability between requirements and tests
- These test cases were executed to verify that the implemented features behave as intended from an end-user perspective.
- For every user story, we tested that all acceptance criteria were met, to check that the story is ‘done’



Unit Testing

- We wrote unit test scripts for each feature and backend API endpoint to verify that individual components work correctly in isolation
- We used pytest for unit testing to help detect regressions early during development
- This ensured that new code changes could be integrated ‘safely’

Integration Testing

- Integration tests were performed to ensure that different modules (e.g. frontend, backend, database) work seamlessly together
- This included testing API calls, database updates, and user interface flows

Test Coverage and Quality

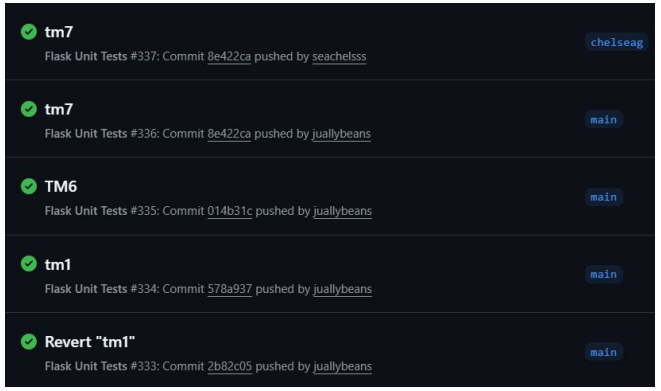
- Combined, our functional and unit tests cover all key user stories and major backend functionalities
- Overall test coverage of 86%



Our testing strategy ensures both requirement coverage (traceable test cases) and code coverage (unit/integration tests), supporting a reliable and maintainable system.

How do we manage our test cases (automated and functional test cases)?

A	B
NavPage / Template	[TGO1] Card
Test Case ID:	TGO1-T1
Acceptance Criteria:	- User can enter a project name, description, and optional tag(s) - Project is saved and visible in the project list
Items	Description
Test Scenario	Verify that a manager can successfully create a new project
Pre-conditions	Logged in as Manager
Test Steps	1. Click "Create Project" 2. Enter project name and description 3. Click "Save"
Test Data	Manager: - Username: george@example.com - Password: 123
Expected Result	New project appears in the project list
Actual Result	Same as expected
Pass/Fail/Not Executed/Blocked	Pass
Remarks	
Created By*	Haroun
Date of Creation*	
Executed By*	Haroun
Date of Execution*	



Automated

- We made use of Github Actions that involved our Python test scripts.
- The test scripts imported the use of pytest and unittest to carry out the automated testing.
- This CI pipeline will automatically check if any new code pushed into a branch passes the tests.
- Any new code that fails the tests is flagged out and notification alerts are sent.
- Our automated test scripts covered all 6 of the core functionalities.
- New test cases were added to the scripts as each core functionality was being further developed.

Functional - Manual

- For each user story within each of the 6 core functionalities, multiple test cases were created based on their acceptance criteria.
- A test case covers the test scenario, pre-conditions, test steps, test data, expected results and actual result.
- The test case metadata includes the people who created and executed it, as well as the dates of creation and execution.

How our process was supported by CI

Our Workflow



- **Branching**: We used modified feature branches ('per-developer' branches).
- **Rationale**: This was a conscious trade-off to avoid the complexity of many feature branches and reduce merge conflicts from shared branches.
- **Merging**: Relied on developer "due diligence" (e.g., local testing) before merging directly to main. We did not use Pull Requests.

Support for Integration:

It acted as our automated safety net. It validated every commit in a clean environment and alerted developers to breakages *before* they merged to main.

CI Pipeline & Integration Support

- **CI Pipeline (`ci.yml`)**: Our pipeline
- provided automated support:
- **Trigger:** on: push to any branch.

```
on:
  push:
```

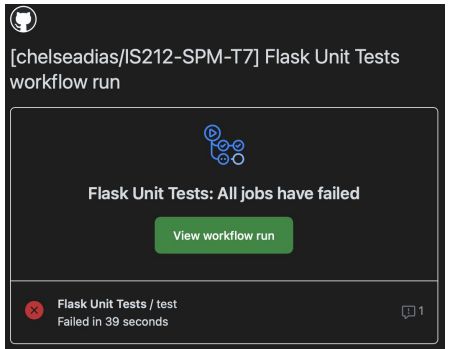
- **Pytest:** Test scripts were co-located with their core function files (e.g., in the same module directory) to clearly associate errors with their source.



- **Test Coverage:** Runs `pytest-cov` to measure coverage and uploads a summary to the GitHub job summary.
- **Alert:** Email on test failure.

```
- name: Run unit tests with coverage
  env:
    FLASK_ENV: testing
  run: |
    pytest --cov=backend --cov-report=term-missing --cov-report=xml -v

- name: Upload coverage to GitHub summary
  if: always()
  run: |
    echo "### Test Coverage Summary" >> $GITHUB_STEP_SUMMARY
    coverage report >> $GITHUB_STEP_SUMMARY
```

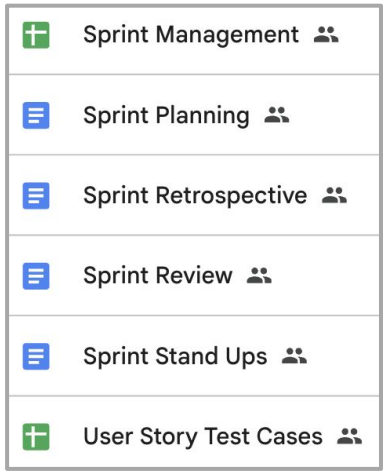


How do we track our progress

1. Scrum Ceremonies

Progress is tracked at every stage:

- **Planning**: Commit to stories in the Sprint Backlog.
- **Standups**: Synchronize work & surface blockers.
- **Review**: Demonstrate and validate completed work.
- **Retrospective**: Analyze performance



2. Key Metrics



We quantify progress using:

- **Story Points (1-8):** Estimated effort (via Scrum Poker Online).
- **Velocity:** Actual points completed per sprint.
- **Definition of Done:** A binary (0% or 100%) measure of completion.

3. Quality & CI

We track technical integration by:

- **Test Traceability:** User Story → Criteria → Test Case ID.
- **Automated CI Pipeline:** Our GitHub Actions pipeline automatically runs all unit tests on every Pull Request.

NavPage / Template	[TM5] Card
[redacted]	[redacted]
Test Case ID:	**TM5-T1**
Acceptance Criteria:	- Managers can remove assignees

4. Tracking Toolkit

Our single sources of truth:

- Sprint Management: For backlogs & user stories.
- Telegram/Slack: For real-time blocker resolution.
- Scrum Poker Online: For unbiased estimation.

